

Supplementary Material for Spill-Cost-Aware Liveness Shaping for NPU Intra-Kernel Scheduling

Anonymous Author

Contents

1	Problem Definition Supplement	2
1.1	DAG and Buffer Lifetimes	2
1.2	Three Evaluation Views	2
1.3	Capacity, Address, and Spill Rules	3
1.4	Benchmark Data Description	3
2	Formal Reasoning and Applicability Boundaries	3
2.1	Clean/Dirty Cost Asymmetry	4
2.2	A Working-Set Certificate over the Near-Optimal Set	4
2.3	A Stability Condition for Belady-Style Eviction	5
2.4	Conditionality of the Overflow-Area Surrogate	6
2.5	Monotonicity of Portfolio Selection	7
3	Method Details and End-to-End Comparison	7
3.1	Candidate-Order Design and the Spill-Separation Principle	7
3.2	P1: Peak Residency and Objective Misalignment	9
3.3	P2: Schedule Order Shapes the Clean/Dirty Composition	9
3.4	P3: Prefetch Window and Compute Overlap	10
3.5	End-to-End Reference Comparison	10
3.6	Analysis of Where the Gaps Come From	11
3.7	Runtime Efficiency	11
4	Additional Experimental Evidence	12
4.1	The Role of Schedule Order	13
4.2	Synthetic Distributions and the Applicability Domain	14
4.3	CP-SAT Oracle	14
4.4	Surrogate Objective and Diagnostic Tools	16
4.5	Controlled Clean/Dirty Ablation	19

Supplement Overview

This supplement expands on the three contributions of the main paper. Section 1 gives the full formalization of the problem definition, fixing the DAG input, the capacity parameters, and the three-stage evaluation protocol. Section 2 provides complete proofs of the paper’s three formal results—the spill-inevitability certificate (Theorem 3.1), Belady-margin stability (Proposition 3.3), and the overflow-area approximation guarantee (Theorem 3.2)—and discusses the applicability boundary of each. Section 3 unfolds the design details compressed in the method section, covering candidate-order generation, the address-assignment procedure, and prefetch-window selection, together with an end-to-end comparison against an external reference solution. Section 4 supplies additional experimental evidence: the separation of schedule order from victim rule, the

applicability validation on a synthetic distribution, the CP-SAT oracle optimality checks, and the controlled clean/dirty ablation.

1 Problem Definition Supplement

The main text (§2) establishes the P1/P2/P3 evaluation framework through a condensed set of definitions. This section supplies the full formal conventions needed to reproduce our experiments, including the DAG fields, the detailed formulas for the evaluation metrics, and the precise parameters of the spill mechanism.

1.1 DAG and Buffer Lifetimes

The main text gives only an outline of the two node categories; here we spell out the complete field definitions. The input is a directed acyclic graph $G = (V, E)$. Its nodes fall into two classes:

1. Operation nodes carry (Id, Op, Pipe, Cycles, BuFs) and represent a compute or data-movement instruction on some hardware pipeline; BuFs lists the logical buffers that the instruction reads and writes.
2. Cache-management nodes carry (Id, Op, BufId, Size, Type), where $\text{Op} \in \{\text{ALLOC}, \text{FREE}\}$. Every logical buffer b has a unique allocation node a_b , a unique free node f_b , a size s_b , and a cache type $\tau(b)$.

Edges encode execution dependencies that must be satisfied. Ignoring the cache-management nodes, a typical sample graph enters through COPY-IN, proceeds through on-chip computation or data movement, and finally writes back off-chip via COPY-OUT; our algorithm, however, does not rely on any specific operator family or motif name. A schedule $S = (v_1, \dots, v_N)$ is a topological order covering all required nodes.

1.2 Three Evaluation Views

The main text condenses P1/P2/P3 into a single paragraph; here we expand their mathematical definitions. The three subproblems add constraints incrementally on the same graph. Let the residency increment that node v contributes be

$$M_v = \begin{cases} +s_b, & v = a_b \text{ (ALLOC)}, \\ -s_b, & v = f_b \text{ (FREE)}, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

P1: peak residency. P1 judges only the node order and requires neither physical addresses nor a spill list. Define the prefix sums

$$V_{\text{stay}}(0) = 0, \quad V_{\text{stay}}(i) = V_{\text{stay}}(i-1) + M_{v_i}, \quad V_{\text{stay}}^{\max} = \max_{0 \leq i \leq N} V_{\text{stay}}(i). \quad (2)$$

The primary objective is to minimize $\max V_{\text{stay}}$. We also report the per-cache peaks, which help explain the mismatch between the P1 order and P2/P3.

P2: extra DDR traffic. Under the capacity constraint, P2 produces physical addresses and a spill list, with the objective of minimizing the extra data movement induced by spilling:

$$\text{extra}(S) = \sum_{b \in \text{Spilled}(S)} e_b, \quad e_b = \kappa_b s_b, \quad \kappa_b = \begin{cases} 1, & b \text{ is clean,} \\ 2, & b \text{ is dirty.} \end{cases} \quad (3)$$

P3: pipeline execution time. After inserting the spill nodes and the address-reuse dependencies, P3 computes the makespan under a multi-pipeline earliest-issue model. Writing $\sigma(v)$ for the start time, $\epsilon(v)$ for the finish time, and E' for the augmented dependency set:

$$\sigma(v) \geq 0, \quad \sigma(v) \geq \epsilon(u) \quad \forall (u, v) \in E', \quad \epsilon(v) = \sigma(v) + \text{Cycles}(v), \quad T = \max_v \epsilon(v). \quad (4)$$

Cache-management nodes have cycle count 0; nodes on the same Pipe execute serially in schedule order, and no reordering within a pipeline is permitted.

Lexicographic keys. We compare schedules using the same lexicographic keys as the main text: P1 uses $(\max \text{live}_{L1}, \max \text{live}_{UB}, L0 \text{ count}, T)$; P2 uses $(\text{extra}, \# \text{spills}, T)$; and P3 uses $(T, \text{extra}, \# \text{spills})$. The benchmark definition provides no single scalar that merges runtime with extra traffic, so for P3 we adopt this lexicographic ordering explicitly as the tie-breaking criterion.

1.3 Capacity, Address, and Spill Rules

The main text briefly states the asymmetric cost of clean/dirty buffers; here we give the full rules for address assignment and the spill mechanism. In P2/P3, a buffer b occupies a contiguous interval $[o_b, o_b + s_b]$ at offset o_b within a single cache. At any instant, the intervals of co-resident buffers belonging to the same cache must not overlap, and they must satisfy $0 \leq o_b$ and $o_b + s_b \leq \text{Cap}_{\tau(b)}$. The capacity parameters are given in Table S1.

Table S1: Hardware cache types and capacities (abstract units).

Cache type	L1	UB	L0A	L0B	L0C
Capacity Cap	4096	1024	256	256	512

We call a buffer b clean if and only if it appears in the Bufs of some COPY-IN node; in that case an off-chip backing copy already exists, so eviction needs no write-back. All other buffers are dirty. The corresponding spill-cycle model is

$$\text{Cycles}(\text{SPILL_IN}_b) = 2s_b + 150, \quad \text{Cycles}(\text{SPILL_OUT}_b) = \begin{cases} 0, & b \text{ is clean,} \\ 2s_b + 150, & b \text{ is dirty.} \end{cases} \quad (5)$$

If the original graph has N nodes, the k -th spill generates a pair of implicit nodes:

$$\text{Id}(\text{SPILL_OUT}_k) = N + 2(k - 1), \quad \text{Id}(\text{SPILL_IN}_k) = N + 2(k - 1) + 1. \quad (6)$$

The SPILL-OUT uses MTE3 and the SPILL-IN uses MTE2. The basic dependency chain for the target buffer b is

$$a_b \rightarrow \text{SPILL_OUT} \rightarrow \text{SPILL_IN} \rightarrow f_b. \quad (7)$$

Every operation node that has already used b must precede the SPILL-OUT, and every operation node that has not yet used b must follow the SPILL-IN. The physical address may change at the SPILL-IN, so feasibility is checked over residency segments rather than over the logical lifetime $a_b \rightarrow f_b$ alone.

Address reuse introduces an additional dependency: if buffer b reuses the physical interval of an earlier buffer a , the edge $f_a \rightarrow a_b$ is added before timing. This dependency leaves the spill traffic unchanged but may alter the P3 makespan.

1.4 Benchmark Data Description

The sizes of the six public NPU intra-kernel scheduling instances used in this work are listed in Table S2. All experiments start from the DAG structure and use no rules specialized to Conv, FlashAttention, or Matmul.

2 Formal Reasoning and Applicability Boundaries

The method section of the main paper grounds spill-cost-aware liveness shaping in three formal results: Theorem 3.1 (the spill-inevitability certificate), Proposition 3.3 (Belady-margin stability), and Theorem 3.2 (the overflow-area approximation). This section gives their complete proofs and states precisely the conditions under which each result applies, so that none of them is overclaimed.

Table S2: Sizes of the public NPU intra-kernel scheduling instances.

Task	Nodes	Edges	Buffers	Op. nodes
Matmul_Case0	4160	7104	1216	1728
Matmul_Case1	30976	55040	8960	13056
FlashAttention_Case0	1716	2712	572	572
FlashAttention_Case1	6952	11184	2328	2296
Conv_Case0	2580	3869	831	918
Conv_Case1	36086	85653	12013	12060

The three results delimit the method’s scope from complementary angles. The spill-inevitability certificate diagnoses capacity limitation through a working-set lower bound taken over the near-optimal set of schedules: when the certificate holds, the objective shifts from eliminating spills to lowering their cost. Belady-margin stability explains why the primary optimization degree of freedom is schedule order rather than eviction policy: under a distance-dominant condition, different Belady-style eviction rules select the same victim set, so any difference between them can stem only from order. The overflow-area approximation theorem then provides a conditional guarantee that the surrogate objective Φ tracks spill traffic in the capacity-bound regime. The following subsections present the complete proofs in turn, keeping “strictly valid theorems,” “conditional propositions,” and “empirical observations” clearly separated.

2.1 Clean/Dirty Cost Asymmetry

Section 2 of the main paper partitions buffers into clean and dirty classes and notes that, although the two occupy identical on-chip space, their spill costs differ by $2\times$. The following observation gives a quantitative basis for this asymmetric structure; it is the starting point for every cost-aware decision that follows.

Observation (clean/dirty cost asymmetry). Let x and y be two buffers of equal size s , with x clean and y dirty. If a spill replaces y by x , the P2 extra traffic of that spill drops from $2s$ to s , that is, by exactly one half.

Deriving the cost ratio. The asymmetry follows directly from the definition of the evaluation metric: by (3), the extra traffic of a clean buffer is s and that of a dirty buffer is $2s$, so their ratio is $1/2$. This is an immediate corollary of the metric itself and does not depend on any particular scheduler or dataset. In the synthetic ablation, holding the DAG, the spill count, and the peak fixed, a clean reserve and a dirty reserve yield extras of 1536 and 3072, respectively, which we use to check that the implementation faithfully reflects this definition.

2.2 A Working-Set Certificate over the Near-Optimal Set

Fix a topological order S and write $\text{pos}_S(v)$ for the position of node v in that order. The logical lifetime of a buffer b is

$$I_S(b) = [\text{pos}_S(a_b), \text{pos}_S(f_b)). \quad (8)$$

The logical live working set of cache c at position t is

$$W_c^S(t) = \sum_{b: \tau(b)=c, t \in I_S(b)} s_b, \quad W_c^S = \max_t W_c^S(t). \quad (9)$$

For a candidate set of orders $\mathcal{S}$, let $E^*(S)$ denote the minimum P2 extra traffic over all legal address assignments, spill choices, and spill placements once the underlying node order S is fixed. Given $\epsilon \geq 0$, define the near-optimal set

$$\mathcal{O}_\epsilon = \{S \in \mathcal{S} : E^*(S) \leq (1 + \epsilon) \min_{S' \in \mathcal{S}} E^*(S')\}, \quad (10)$$

together with the working-set lower bound over that set,

$$\underline{W}_c^\epsilon = \min_{S \in \mathcal{O}_\epsilon} W_c^S. \quad (11)$$

Theorem 3.1 of the main paper asserts that if the working-set lower bound over the near-optimal extra-traffic subset exceeds the cache capacity, then spilling is unavoidable. We now give the complete proof and discuss the sense in which this is a one-sided condition: it can confirm that spilling is unavoidable, but the converse does not hold.

Theorem 3.1 (spill inevitability). If $\underline{W}_c^\epsilon > \text{Cap}_c$, then every $S \in \mathcal{O}_\epsilon$ must incur a capacity spill on cache c . Equivalently, within the candidate range that preserves near-optimal P2 extra traffic, address placement alone cannot make cache c entirely spill-free. More quantitatively, for any $S \in \mathcal{O}_\epsilon$ and any legal P2 implementation P respecting S ,

$$\text{extra}(P) \geq \max_t (W_c^S(t) - \text{Cap}_c)_+. \quad (12)$$

Proof. At any position t , decompose the logical live volume into the resident volume $R_c^P(t)$ and the volume that has been spilled and is momentarily absent, $A_c^P(t)$:

$$W_c^S(t) = R_c^P(t) + A_c^P(t). \quad (13)$$

The capacity constraint gives $R_c^P(t) \leq \text{Cap}_c$, hence

$$A_c^P(t) \geq (W_c^S(t) - \text{Cap}_c)_+. \quad (14)$$

Any buffer that is momentarily absent yet still live must have been spilled at least once before this position, and a single spill contributes extra traffic at least equal to that buffer’s size. Therefore the total extra traffic is no smaller than the absent live volume at any instant. Taking the maximum over t yields the displayed bound. Finally, if $\underline{W}_c^\epsilon > \text{Cap}_c$, then every order in $\mathcal{O}_\epsilon$ has some position at which $W_c^S(t) > \text{Cap}_c$, so spilling cannot be entirely avoided on that cache.

One-sidedness and scope of the certificate. The theorem provides a one-sided certificate: it can confirm only that spilling is unavoidable, and the converse does not hold. When $\underline{W}_c^\epsilon \leq \text{Cap}_c$, one cannot conclude spill-freedom; fragmentation, address-reuse dependencies, pinned resident segments, and reload positions may still force spills. The restriction to the near-optimal set is also indispensable: if one minimizes the peak over all legal orders without constraint, certain orders with very poor P2 extra traffic but low peaks can make an instance appear “not capacity-bound.” For example, the full-order minimum working-set ratio for Matmul_Case1/L1 is 0.03125, whereas within the 10% near-optimal extra-traffic set the ratio is 8.5; it is the latter that reflects the capacity pressure in the P2-relevant region. The CP-SAT oracle validation in §4.3 of the main paper confirms that this lower bound holds across all small graphs.

2.3 A Stability Condition for Belady-Style Eviction

Proposition 3.3 of the main paper states that, under a distance-dominant condition, different Belady-style eviction rules select the same victim set. We now give the formal condition and its proof.

Fix a schedule order. At a cache-overflow decision k , let R_k be the set of evictable resident buffers, Δ_k the volume that must be freed, and $d_k(b)$ the next-use distance of buffer b . Consider a family of distance-dominant Belady-style rules

$$\text{score}_r(b) = d_k(b)g_r(b), \quad (15)$$

where $g_r(b) > 0$ is a bounded, rule-dependent perturbation, such as a clean/dirty cost weight. Assume that all rules and candidates satisfy $g_r(b) \in [g_{\min}, g_{\max}]$, and write $\rho = g_{\max}/g_{\min}$.

Proposition 3.3 (Belady-margin stability). Suppose there is a set $F_k \subseteq R_k$ satisfying: (i) $\sum_{b \in F_k} s_b \geq \Delta_k$; (ii) no proper subset of F_k suffices to free Δ_k ; and (iii) every $x \in F_k$ and $y \in R_k \setminus F_k$ satisfy

$$d_k(x) > \rho d_k(y). \quad (16)$$

Then all the rules above prefer the same victim set F_k , possibly differing only in the tie order within F_k . If this condition holds at every overflow decision, then these rules produce the same residency evolution and the same P2 extra traffic.

Proof. For any rule r , $x \in F_k$, and $y \notin F_k$,

$$\frac{\text{score}_r(x)}{\text{score}_r(y)} = \frac{d_k(x)g_r(x)}{d_k(y)g_r(y)} \geq \frac{d_k(x)g_{\min}}{d_k(y)g_{\max}} > 1. \quad (17)$$

Hence every element of F_k ranks ahead of every element outside F_k . A greedy selection that picks the highest-scoring elements until enough volume is freed therefore selects exactly F_k , by sufficiency and minimality.

Intuition and applicability. The cost-weighted distance score used in this work is $d_k(b)s_b/e_b$. Since $e_b = s_b$ or $2s_b$, the clean/dirty weight induces a perturbation of at most $2\times$. Consequently, when the next-use distance of a far-future candidate exceeds twice that of a near-term candidate, the cost weight cannot change the victim set. If the candidates are approximately homogeneous in size and clean/dirty class, different Belady-style rules likewise degenerate to the same ordering. This proposition explains why the victim rule is often insensitive on a fixed order, but it is not an unconditional theorem: if short-lived clean buffers and long-lived dirty buffers alternate repeatedly, the rule differences can accumulate. The controlled experiments in §4.3 of the main paper confirm this prediction: on a fixed order, the extra traffic of four victim rules differs by $\text{CV} < 4\%$, whereas the swing in extra traffic across different orders can reach $10\times$ or more.

2.4 Conditionality of the Overflow-Area Surrogate

Let the unnormalized overflow area be

$$A(S) = \sum_t \sum_c (W_c^S(t) - \text{Cap}_c)_+, \quad (18)$$

and the normalized surrogate be

$$\Phi(S) = \sum_t \sum_c \frac{(W_c^S(t) - \text{Cap}_c)_+}{\text{Cap}_c}. \quad (19)$$

Both measure the “over-capacity memory-time area.” P2 extra traffic, by contrast, is an event cost: a single spill can keep a buffer absent across many schedule positions. Consequently, there is no unconditional constant-factor approximation between $A(S)$ (or $\Phi(S)$) and the optimal extra traffic. A long-lived small overflow makes the area grow with time while the extra traffic is paid only once; conversely, a large buffer that exceeds capacity by only one unit may nonetheless force the eviction of an entire large block.

Theorem 3.2 of the main paper establishes a two-sided constant relation between the overflow area $A(S)$ and the optimal extra traffic under a bounded-absence-duration assumption. We now give the complete proof together with a counterexample construction.

Theorem 3.2 (conditional overflow-area approximation). Fix an order S . Suppose there exists a legal spill plan P whose every spill episode has absence duration ℓ_j satisfying $\ell \leq \ell_j \leq L$, whose removed memory-time area

$$R(P) = \sum_j s_j \ell_j \quad (20)$$

satisfies $R(P) \leq \lambda A(S)$, and whose every unit spill cost κ_j satisfies $\kappa_{\min} \leq \kappa_j \leq \kappa_{\max}$. Then the fixed-order optimal extra traffic satisfies

$$\frac{\kappa_{\min}}{L} A(S) \leq E^*(S) \leq \frac{\kappa_{\max} \lambda}{\ell} A(S). \quad (21)$$

In the evaluation model of this work, $\kappa_{\min} = 1$ and $\kappa_{\max} = 2$.

Proof sketch. Any legal implementation must remove at least $(W_c^S(t) - \text{Cap}_c)_+$ of volume at each position, so $A(S) \leq R(P)$. Since $\ell_j \leq L$, we have $R(P) \leq L \sum_j s_j$; as each unit of extra traffic costs at least $\kappa_{\min}$, this yields the lower bound. For the upper bound we use the feasible plan assumed in the hypothesis: from $\ell_j \geq \ell$ we obtain $\sum_j s_j \leq R(P)/\ell \leq \lambda A(S)/\ell$, which, multiplied by the maximum per-unit cost $\kappa_{\max}$, gives the upper bound.

From A to Φ . Because each cache capacity is fixed, with minimum capacity 256 and maximum capacity 4096,

$$256 \Phi(S) \leq A(S) \leq 4096 \Phi(S). \quad (22)$$

Stating a direct theoretical guarantee in terms of Φ therefore incurs an additional $16\times$ capacity-span factor. The main reason this work uses Φ is not that it is empirically much stronger than per-cache peak pressure, but that it provides a lifetime-aware, cross-cache-comparable candidate-selection signal at $O(N)$ cost. The empirical data reported in §4.3 of the main paper corroborate this positioning: over the evaluation set, the global Spearman correlation between Φ and P2 extra traffic is 0.958, against 0.955 for per-cache peak pressure, so the two are essentially on par; and the empirical ratio of Theorem 3.2, $E^*/A \in [0.56, 1.13]$, falls within the constant-factor band. This fact also requires restraint in the main paper: the value of Φ lies in computational efficiency and cross-stage consistency, not in predictive accuracy significantly better than per-cache peak.

2.5 Monotonicity of Portfolio Selection

The method section of the main paper (§3.2) notes that adding candidate orders cannot degrade the result. We give the formal argument here. The safety of the candidate portfolio follows from a simple fact. Let $\mathcal{P}$ be a finite set of candidate orders, victim rules, or prefetch windows, and let $J(x)$ be the true evaluation key (which may be a lexicographic key). If the output is chosen by $\arg \min_{x \in \mathcal{P}} J(x)$, then adding a new candidate z gives

$$\min_{x \in \mathcal{P} \cup \{z\}} J(x) \leq \min_{x \in \mathcal{P}} J(x). \quad (23)$$

Hence, as long as the final selection uses the true P2/P3 evaluation key rather than a surrogate score, enlarging the candidate set cannot degrade the selected result; the surrogate Φ is used only to generate and screen candidates and does not replace the final objective.

3 Method Details and End-to-End Comparison

Section 3 of the main text gives the overall structure of candidate-order generation and address assignment as algorithmic pseudocode. This section expands on three aspects of the design: the rationale for the complementary candidate orders (§3.1), the end-to-end results compared against the external reference solution (§3.2–§3.5), and a diagnostic analysis of where the gaps come from (§3.6).

3.1 Candidate-Order Design and the Spill-Separation Principle

The main text treats schedule order as the primary degree of freedom for optimizing spill cost. This strategy requires the candidate-order set to cover the different clean/dirty compositions that can arise within peak windows, rather than merely chasing the local optimum of a single metric. We first explain why the three candidate orders are complementary, and then discuss the separation of the prefetch window from spill selection.

Candidate orders. All three candidate orders are structure-agnostic topological orders. They share the same memory-aware list scheduling skeleton (frees-first, allocations-delayed) and differ only in how they rank the ready set.

1. The pressure-aware order (*p1*) is the memory-aware list scheduling order used by Algorithm 1 of the main text, applying a memory-aware priority within each ready set. ALLOC nodes are ranked by successor indegree / successor readiness: a lower indegree means the consumer becomes ready sooner, so the allocation is deferred until close to its use, pushing the allocation timing to the point least sensitive to the peak. This order is the direct output of P1 and is also one candidate for P2/P3.
2. The capacity-throttled order (*capfit-id*) retains the memory-aware free/operation priority within each ready set, but ranks ALLOC nodes by minimum node identifier instead, and overlays capacity gating on top of this base order: if an ALLOC would push its cache over capacity, the allocation is held back; when all ready allocations are blocked and no further deferral is possible, the one that unblocks subsequent work the fastest is forced through (with weights normalized by $1/\text{Cap}_c$). The role of this order is to directly bound the peak height, at the cost of sacrificing some of the continuous residency of buffers.
3. The ID-reserve order (*id-raw*) always takes the minimum identifier within the free, operation, and allocation ready sets, with neither a memory-aware priority nor capacity gating. Its purpose is to preserve the original residency order of clean COPY-IN buffers, making these clean buffers more likely to stay resident through the compute window and thereby leaving a clean reserve in high-pressure windows that can be evicted at half price. This effect cannot be reliably produced by the first two orders, since both of their ranking strategies disturb the original order of the input buffers.

The complementarity of the three orders manifests as distinct paths for shaping the clean/dirty composition: the capacity-throttled order (*capfit-id*) lowers the total peak through capacity gating, the pressure-aware order (*p1*) shortens active intervals through delayed allocation, and the ID-reserve order (*id-raw*) maintains a clean reserve through order preservation. As noted in Section 3.2 of the main text, candidate-set expansion is monotone: adding a new candidate can only improve or preserve the selected objective.

Address assignment and cost-aware eviction. Given a topological order, best-fit address assignment is performed along the order. When capacity is insufficient, the victim is chosen by a cost-weighted Belady score:

$$\text{victim} = \arg \max_b d(b) \frac{s_b}{e_b} = \arg \max_b \begin{cases} d(b), & b \text{ clean}, \\ d(b)/2, & b \text{ dirty}, \end{cases} \quad (24)$$

where $d(b)$ is the next-use distance of buffer b . The effective distance of a dirty buffer is halved, so the engine prefers to evict the cheaper clean buffers: a dirty buffer is evicted ahead of a clean buffer of the same size only when its future use is far enough away. Proposition 3.3 of the main text shows that under distance-dominant conditions this scoring rule selects the same eviction set as other cost-weighted variants, so the marginal benefit of the eviction policy is far smaller than that of schedule order.

Separating the prefetch window from spill selection. The selection of the spill set and the placement of reloads are two separable degrees of freedom. The former determines the total extra; the latter affects whether reloads can overlap with computation, and hence the P3 time. Coupling the two would let time optimization contaminate the traffic objective without constraint. The method’s design choice is to keep them apart. The prefetch window H controls only the advancement of SPILL-IN: if an evicted buffer will be used within $\leq H$ steps and the cache has free space, it is reloaded early, but no extra eviction is triggered to enable the prefetch. P2 uses the tight grid $H \in \{0, 5, 40\}$ to minimize traffic; P3 uses the wider grid $H \in \{0, 5, 40, 80, 120\}$ to minimize time.

Selection mechanism. A full simulation is run over each point of the Cartesian product of the 3 candidate orders and the prefetch windows: P2 selects by the lexicographic key (extra, #spills, T), and P3 by (T , extra, #spills). The final output is the minimal item over the candidate set.

Algorithm 1 Unified solver framework for P1/P2/P3

Require: DAG $G = (V, E)$, problem type $p \in \{1, 2, 3\}$

Ensure: schedule; for P2/P3 also output address assignment mem and spill list $spill$

```

1: if  $p = 1$  then
2:   return MemoryAwareOrder( $G$ )
3: end if
4:  $\mathcal{H} \leftarrow \{0, 5, 40\}$  if  $p = 2$  else  $\{0, 5, 40, 80, 120\}$ 
5:  $best \leftarrow \perp$ 
6: for  $\sigma \in \{capfit-id, p1, id-row\}$  do
7:   for  $H \in \mathcal{H}$  do
8:      $(\pi, mem, spill) \leftarrow \text{AssignAndSpill}(G, \sigma, H)$ 
9:      $E \leftarrow \sum_{(b, \cdot) \in spill} e_b$ ;  $T \leftarrow \text{SimTime}(\pi, mem, spill)$ 
10:     $k \leftarrow (E, |spill|, T)$  if  $p = 2$  else  $(T, E, |spill|)$ 
11:     $best \leftarrow \min(best, (k, \pi, mem, spill))$ 
12:   end for
13: end for
14: return  $best$ 

```

3.2 P1: Peak Residency and Objective Misalignment

P1 is judged on per-cache peak residency; its result directly answers “whether a given DAG can lower its peak under some legal order.” However, a P1-optimal order does not automatically translate into a P2/P3 optimum, which is precisely the objective misalignment phenomenon emphasized in the main text.

The end-to-end results show that our order lowers the L1 peak below the external reference solution on all six instances: Conv_Case0 drops from 7488 to 6912, and Conv_Case1 from 14040 to 13786; the reductions are even larger on the FlashAttention and Matmul instances, where the P1 peaks fall to 256/256/128/128. But a low P1 peak is not equivalent to a low spill cost. Take FlashAttention_Case1 as an example: the P1 order drives the UB peak to 33010, far exceeding $\text{Cap}_{UB} = 1024$. The order that lowers the L1 peak instead causes a more severe capacity overrun on UB. This case directly explains why the method must move from the single-peak P1 metric to the spill-traffic metric of P2: in a multi-cache system, it is the cross-cache distribution of peaks that determines the true spill cost.

3.3 P2: Schedule Order Shapes the Clean/Dirty Composition

The core objective of P2 is to minimize the total spill traffic extra. Unlike swapping only the eviction policy on a fixed order, our approach uses schedule order to change the composition of clean and dirty buffers in high-pressure windows, so that when capacity is exceeded the evictor can free space at a cost of s_b (clean) rather than $2s_b$ (dirty).

Relative to the external reference solution, P2 wins three and loses three of the six instances. The wins reveal the effectiveness of the shaping mechanism: in Conv_Case1 (73240 $\rightarrow$ 72520), our order keeps more clean buffers in the high-pressure window; Matmul_Case0 (34944 $\rightarrow$ 34688) behaves likewise; and on Matmul_Case1 both the extra and the spill count are level with the reference solution (460800, 3600), but the third key, time, is better. The three losing instances point to the flip side of the same mechanism: in the critical high-pressure window, the reference order happens to retain more suitable clean buffers. For instance, in Conv_Case0 the reference solution’s extra is 73500, below our 88044; the gap does not come from a difference in the eviction rule (Proposition 3.3 shows that different eviction variants on a fixed order differ very

little in extra traffic), but rather from the reference order keeping more clean buffers resident when spilling occurs—exactly the shaping effect of schedule order on the residency composition.

The wins and losses together corroborate the central thesis of the main text: the primary determinant of spill cost is how the order shapes the clean/dirty composition, not the eviction rule itself.

3.4 P3: Prefetch Window and Compute Overlap

P3 takes pipelined execution time as its primary objective. A nonzero prefetch window advances some SPILL-IN operations ahead of their immediate use points, allowing reloads to overlap with computation or other pipeline transfers and reducing the wait-blocking of consumer nodes. Because prefetching occurs only when idle intervals exist and triggers no extra eviction, it mainly adjusts the timing of reload placement rather than redefining the spill set.

The end-to-end results are four wins and two losses. Our method achieves shorter times on the two FlashAttention and two Matmul instances: FlashAttention_Case1 drops from 193059 to 180364, and Matmul_Case0 from 194331 to 186820. Our times lag on the two Conv instances, and Conv_Case1 is especially diagnostic: our extra is lower ($73578 < 86690$), yet the reference solution’s time is shorter ($1073322 < 1111778$). This counterexample shows that lower traffic does not automatically translate into shorter time: the degree of overlap between reloads and computation is an independent dimension. When the reference order’s reloads happen to be scheduled into idle gaps in the compute pipeline, the blocking time can be shorter even when the total traffic is higher. This observation further supports the method’s design decision to separate spill selection from reload placement.

3.5 End-to-End Reference Comparison

Table S3 summarizes the end-to-end results of the six public instances on P1/P2/P3, for a combined 13/18 wins. The comparison protocol here differs from that of the main text’s primary experiments: Section 4.2 of the main text uses the “same-engine reorder” protocol, in which all methods share the same address assignment / spill engine and only the schedule order is swapped, so as to isolate the shaping effect of order on the clean/dirty composition. The end-to-end comparison, by contrast, uses each system’s own complete schedule, address assignment, and spill list, reflecting the final output quality of two complete systems. The two protocols answer different questions, and the end-to-end data serve as an overall reference for system performance.

Table S3: End-to-end comparison of our method against the external reference solution baseline01 (each cell is ours/reference).

Case	P	max _{L1}	spills	extra	T	Result
Conv_Case0	P1	6912/7488	0/0	0/0	394430/359570	Win
	P2	6912/7488	339/141	88044/73500	615694/535312	Loss
	P3	6912/7488	342/141	92652/73500	553280/535312	Loss
Conv_Case1	P1	13786/14040	0/0	0/0	650044/573395	Win
	P2	14040/14040	1963/1828	72520/73240	1159169/1150333	Win
	P3	14040/14040	1951/1290	73578/86690	1111778/1073322	Loss
FA_Case0	P1	256/3328	0/0	0/0	31063/31429	Win
	P2	3328/3328	31/41	3904/3692	51700/53374	Loss
	P3	4096/3328	58/24	5832/4128	46167/46761	Win
FA_Case1	P1	256/5376	0/0	0/0	112458/113445	Win
	P2	5376/5376	229/146	32920/32840	214361/193059	Loss
	P3	5376/5376	229/146	34432/32840	180364/193059	Win
Matmul_Case0	P1	128/9216	0/0	0/0	82308/82773	Win
	P2	9216/9216	271/273	34688/34944	192380/194331	Win
	P3	9216/9216	273/273	34944/34944	186820/194331	Win
Matmul_Case1	P1	128/34816	0/0	0/0	629076/583525	Win
	P2	34816/34816	3600/3600	460800/460800	1783145/1800218	Win
	P3	34816/34816	3601/3600	460928/460800	1771132/1800218	Win

3.6 Analysis of Where the Gaps Come From

The core finding across the five losing entries is consistent: the reference solution’s advantage on these instances does not weaken the main text’s thesis but instead corroborates the same mechanism from the opposite direction, namely that the gap comes mainly from how schedule order shapes the clean/dirty residency composition, not from a difference in the eviction policy on a fixed order.

Two kinds of diagnostics cross-validate this judgment. First, when the reference solution’s order is fed into our address assignment / spill engine, the traffic of several losing entries can be closely reproduced, showing that the gap comes from the composition of evictable buffers in the high-pressure window rather than from a difference in the engine implementation. Second, swapping among four Belady-style victim policies on a fixed order produces only small differences in P2 extra (Section 4.3 of the main text reports $CV \leq 4\%$); by contrast, extra can differ by an order of magnitude across different legal orders.

We keep these losing entries rather than fitting them with operator-family specialization rules, because such fitting would break the method’s structure-agnostic property: the algorithm uses only the DAG dependencies, buffer sizes, cache types, clean/dirty attributes, and capacity information, and contains no ranking rules tailored to operator categories. Once specialization is introduced, the conclusion would degrade from “schedule order shapes the clean/dirty composition to lower spill cost” to “a dedicated ordering for a specific operator structure,” weakening the generality of the method.

3.7 Runtime Efficiency

Table S4 gives the complete statistics over 3 repetitions (median, min, max), supplementing the median data of Table 3 in the main text.

Dominant factors in runtime. P1 generates only a single schedule order, and all instances complete in under 0.2 seconds. The runtime of P2/P3 is dominated by two factors: (1) the grid size of candidate orders $\times$ prefetch

Table S4: Wall-clock runtime (seconds, 3 repetitions).

Instance	P	Median	Min	Max	Nodes
Conv_Case0	P1	0.009	0.009	0.010	2 580
	P2	0.391	0.390	0.392	
	P3	0.683	0.677	0.706	
Conv_Case1	P1	0.189	0.186	0.190	36 086
	P2	57.97	57.46	58.62	
	P3	73.14	73.10	73.28	
FA_Case0	P1	0.006	0.006	0.007	1 716
	P2	0.220	0.218	0.222	
	P3	0.328	0.326	0.328	
FA_Case1	P1	0.027	0.026	0.030	6 952
	P2	2.723	2.723	2.757	
	P3	3.356	3.316	3.367	
Matmul_Case0	P1	0.014	0.014	0.016	4 160
	P2	0.701	0.695	0.703	
	P3	1.137	1.136	1.170	
Matmul_Case1	P1	0.126	0.111	0.126	30 976
	P2	19.40	19.33	19.46	
	P3	32.33	32.19	32.34	

windows (P2 evaluates $3 \times 3 = 9$ combinations, P3 evaluates $3 \times 5 = 15$ combinations); and (2) the quadratic worst-case cost of Belady eviction selection in address assignment. Conv_Case1 and Matmul_Case1 have 12 013 and 8 960 buffers respectively, and the eviction scan constitutes the bulk of their runtime. The min/max values over the three repetitions tightly bracket the median (coefficient of variation $< 2\%$), indicating that the timing results are stable.

Time complexity. Let $N = |V|$ be the number of nodes, $E = |E|$ the number of edges, B the number of buffers, $K = 3$ the number of candidate orders, and W the size of the prefetch-window grid. The complexity of each stage is as follows:

- **Candidate-order generation:** $O(K(N \log N + E))$, a heap-maintained ready-set topological sort.
- **Address assignment and spill insertion:** $O(N + E + B^2)$ per (order, window) combination. The sweep along the order is linear, and the worst-case cost of Belady eviction is B allocations each scanning $O(B)$ resident buffers.
- **Pipeline simulation:** $O(N + E)$ per combination.
- **Portfolio selection:** enumeration of $O(KW)$ combinations.

The total is $O(KW(N \log N + E + B^2))$. Since K and W are small constants, the dominant asymptotic term is $O(N \log N + E + B^2)$. The space complexity is $O(N + E + B)$.

4 Additional Experimental Evidence

The experiments section of the main paper retains only the figures and tables that most directly support the central argument. This section supplies finer-grained evidence, organized by the role each piece of evidence plays in the argument: same-engine reorder comparisons and sensitivity analyses isolate the “schedule order” degree of freedom; synthetic distributions delineate the applicability domain of the method; a CP-SAT oracle validates the theoretical lower bound and fixed-order optimality; surrogate-objective, working-set, and

prefetch-window experiments explain the algorithm design; and the clean/dirty ablation isolates the $2\times$ cost gradient.

4.1 The Role of Schedule Order

The controlled comparison in §4.3 of the main text shows that victim-rule differences amount to at most 4% under a fixed order, whereas the extra traffic can swing by an order of magnitude across different orders. This section quantifies that finding through three complementary experiments: a same-engine reorder comparison, a victim-rule sensitivity analysis, and a visualization of the cross-order swing.

Table S5 gives the controlled P2 comparison on six public instances. All methods share the same address assignment and spill engine and differ only in the input topological order. The differences in extra traffic can therefore be attributed to schedule order reshaping the residency composition inside high-pressure windows, rather than to a different allocator or a different victim policy.

Table S5: Controlled comparison: P2 spill traffic (same address assignment and spill engine; only the schedule order is replaced).

Case	CP-list	CP-free	pressure	G–Hsu	ours
Conv_Case0	694,506	212,956	207,932	210,844	88,044
Conv_Case1	1,695,264	73,348	1,048,524	1,053,484	72,520
FA_Case0	211,784	3,904	101,356	90,092	3,904
FA_Case1	965,944	33,152	445,876	418,228	32,920
Matmul_Case0	823,168	34,944	296,064	298,240	34,688
Matmul_Case1	6,506,240	460,800	2,556,928	2,560,640	460,800

The clean/dirty-blind memory-pressure schedulers (uniform pressure and Goodman–Hsu) are substantially worse than our order on every real case, paying roughly $2.4\text{--}26\times$ more extra traffic. The critical-path-oriented CP-list order is further behind still. By contrast, CP-free is a strong companion: by prioritizing node releases it indirectly shortens live ranges, approaching our order on most combinations, yet it remains clearly behind on instances such as Conv_Case0 where clean reserve is decisive for cost. This result shows that reducing live bytes or releasing early is not by itself sufficient to explain the full benefit; the difference in clean/dirty composition inside high-pressure windows is a necessary factor.

To probe the marginal effect of the victim rule directly, Figure S1 replaces four Belady-style victim rules under a fixed schedule order and compares the resulting change in extra traffic per case. The four rules coincide exactly on most Matmul and FlashAttention instances; the Conv rows show visible differences, but their magnitude remains far below the cross-order swing. This agrees with the theoretical prediction of Proposition 3.3: under distance-dominant conditions, the marginal benefit of the victim rule is limited.

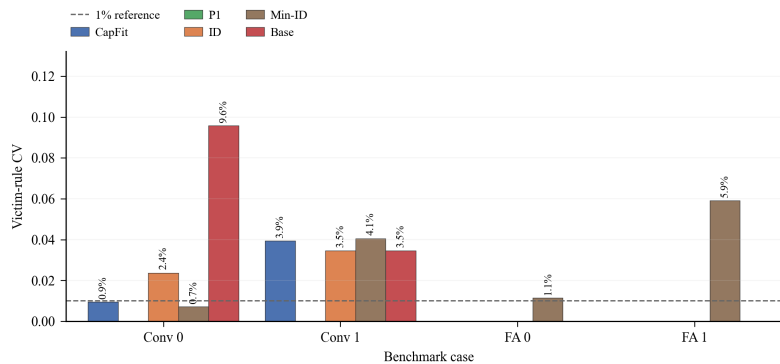


Figure S1: Victim-rule sensitivity under a fixed schedule order. Most Matmul and FlashAttention rows coincide exactly; the Conv rows show visible differences, but their magnitude remains smaller than the cross-order swing.

Conversely, Figure S2 shows the swing in extra traffic across different legal topological orders under the same engine. Replacing the legal topological order substantially changes the composition of evictable buffers in high-pressure windows, and therefore produces a far larger gap in extra traffic than fine-tuning the victim rule. Together, these two experiments confirm from both sides that the victim rule is only a local patch, while schedule order is the principal degree of freedom driving spill cost.

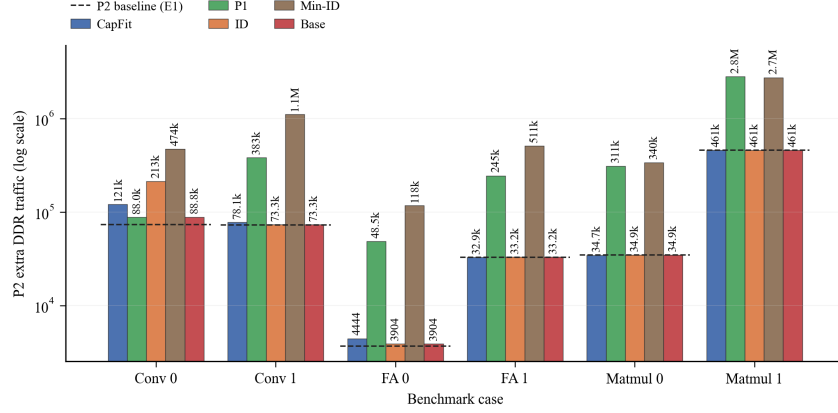


Figure S2: Schedule-order sensitivity. Compared with fine-tuning the victim rule on a fixed order, replacing the legal topological order substantially changes the composition of evictable buffers in high-pressure windows, and therefore produces a larger swing in extra traffic.

Taken together, the three pieces of evidence support the claim that schedule order is the principal optimization lever. Table S5 quantifies the same-engine reorder gap, Figure S1 shows that the marginal effect of the victim rule is limited under a fixed order, and Figure S2 gives the order-of-magnitude difference of the cross-order swing. The gap in spill cost thus comes mainly from how schedule order shapes the clean/dirty residency composition in peak windows, and not from anything a victim rule can make up for on its own.

This paper uses two comparison protocols. Table S5 is the same-engine reorder comparison, used to isolate the scientific mechanism; Table S3 is the end-to-end reference comparison, comparing the complete output of each system. The former supports the claim that schedule order shapes residency composition, the latter reflects the submission quality of the final solver on public instances, and the two should not be conflated.

4.2 Synthetic Distributions and the Applicability Domain

Figure 6 in the main text shows the systematic advantage in the capacity-bound region. This section gives the complete per-instance data and regime statistics over all 36 synthetic instances.

Figure S3 and Table S6 report the distributional results over the 36 synthetic instances. We partition the instances into capacity-bound and order-reachable classes: the former require spilling unavoidably under capacity pressure, while the latter leave room to avoid spills directly through a good order. This partition corresponds to how the spill-inevitability certificate is used in the main text: it is not a complete feasibility test, but a way to identify the region in which the composition of unavoidable spills must be optimized.

The results match the theoretical positioning. In the capacity-bound subgroup, our method wins against both CP-list and random-best on every instance, with a median $2\times$ advantage over CP-free. In the order-reachable subgroup, a good order can eliminate spills directly, so clean/dirty shaping naturally no longer offers a systematic advantage; here the win rate over CP-free drops to 0%. The goal of the method is therefore not to dominate all DAGs, but to lower the real cost per unit of spill when spills are unavoidable.

4.3 CP-SAT Oracle

§4.3 of the main text reports the summary conclusions from the CP-SAT oracle. This section gives the per-instance data, including the capacity-constraint ratio, the theoretical lower bound, and the fixed-order optimal

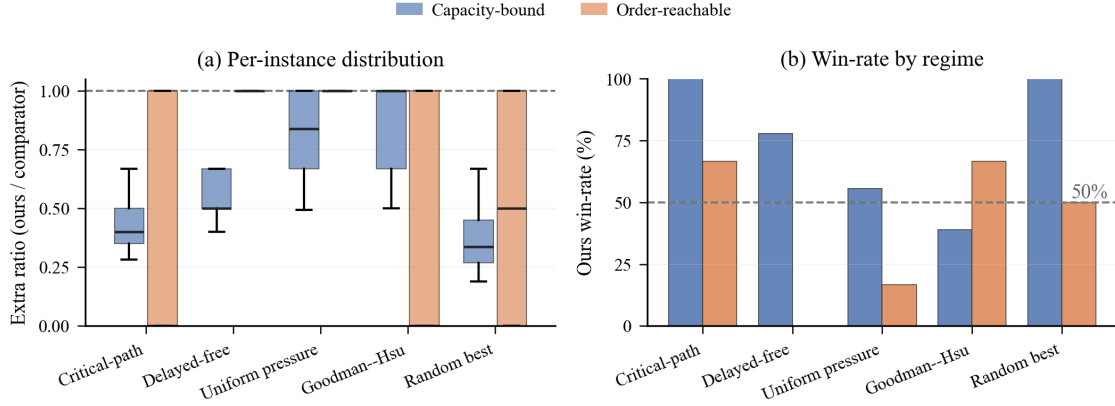


Figure S3: Distribution over synthetic instances. (a) Per-instance extra-traffic ratio; (b) win rate by capacity-bound and order-reachable regime. The advantage concentrates in the capacity-bound region.

Table S6: Distribution statistics over the synthetic suite. The median ratio is ours/comparator; lower values indicate that our order is better.

Comparator	Regime	Median ours/comp	Win rate
<i>CP-list</i>	all	0.39	83.3%
	capacity_bound	0.40	100%
	order_reachable	0.00	66.7%
<i>CP-free</i>	all	1.00	38.9%
	capacity_bound	0.50	77.8%
	order_reachable	1.00	0%
<i>Pressure</i>	all	1.00	36.1%
	capacity_bound	0.84	55.6%
	order_reachable	1.00	16.7%
<i>G-Hsu</i>	all	0.67	52.8%
	capacity_bound	1.00	38.9%
	order_reachable	0.00	66.7%
<i>Random-best</i>	all	0.34	75.0%
	capacity_bound	0.34	100%
	order_reachable	0.50	50.0%

extra traffic. The small-graph oracle answers two questions: given the order our method selects, does the address/spill engine reach the capacity-only optimum; and do the theoretical lower bound and the area surrogate remain consistent on exact solutions.

Table S7 reports results for 8 capacity-bound small graphs. $\text{ratio}_{W^*} = W^*/\text{Cap} > 1$ indicates that the instance is capacity-bound; lb_{T_1} is the lower bound given by the spill-inevitability certificate of the main text; and E^* is the optimal extra traffic under a fixed order.

On all small graphs, the gap of our engine relative to the fixed-order optimum is 1.0, showing that for this set of oracles the bottleneck lies in order selection rather than in the allocation/eviction implementation. The spill-inevitability lower bound holds throughout, and $E^*/A \in [0.56, 1.13]$, falling within the constant-factor range described in the main text. A cross-order joint-optimum model would have to couple the permutation variables with the residency variables, which causes CP-SAT to time out frequently; this paper therefore makes no claim about the gap to the global optimum, and uses the fixed-order oracle only to validate local optimality and the theoretical bounds.

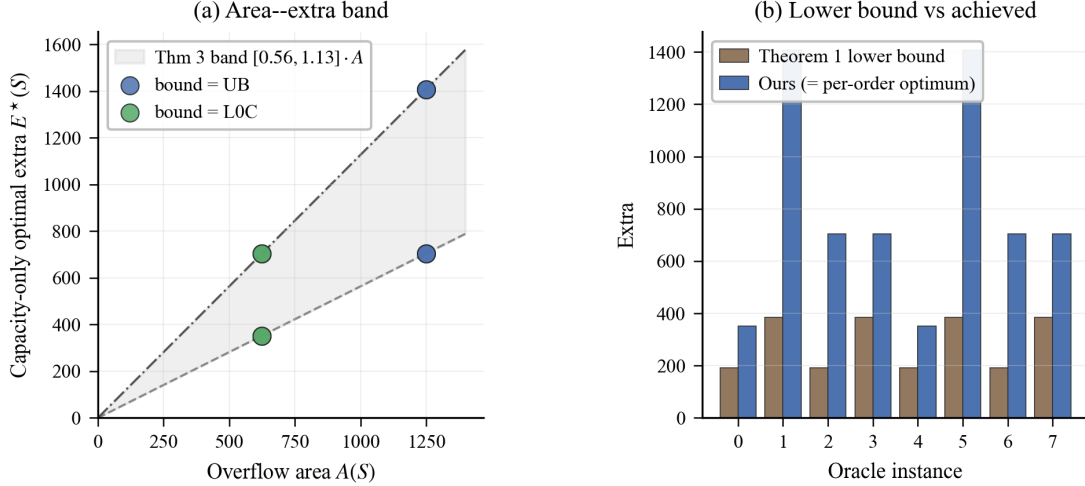


Figure S4: CP-SAT oracle validation. (a) Empirical constant band between overflow area $A(S)$ and fixed-order optimal extra traffic $E^*(S)$; (b) the spill-inevitability lower bound against the fixed-order optimal extra traffic.

Table S7: Per-instance oracle data. All instances satisfy $lb_{T1} \leq E^*$, and our engine reaches E^* under a fixed order.

inst	bound	Cap	ratio $_{W^*}$	lb_{T1}	E^*	E^*/A
oracle_000	LOC	512	1.375	192	352	0.564
oracle_001	UB	1024	1.375	384	1408	1.128
oracle_002	LOC	512	1.375	192	704	1.128
oracle_003	UB	1024	1.375	384	704	0.564
oracle_004	LOC	512	1.375	192	352	0.564
oracle_005	UB	1024	1.375	384	1408	1.128
oracle_006	LOC	512	1.375	192	704	1.128
oracle_007	UB	1024	1.375	384	704	0.564

4.4 Surrogate Objective and Diagnostic Tools

§4.3 of the main text reports, in summary form, the experimental conclusions for the overflow-area surrogate, the working-set certificate, and the prefetch window. This section gives the per-instance data and visualizations for these diagnostic tools, together with a quantitative analysis of the P1 versus P2/P3 misalignment.

Overflow-area surrogate. Figure S5 compares the explanatory power of the overflow-area surrogate Φ and per-cache peak pressure for P2 extra traffic. Their Spearman correlations are very close (0.958 and 0.955). We therefore do not present Φ as an empirically much stronger predictor; its value lies in being cheap to compute, in being usable stably during candidate-order filtering, and in being consistent with the area-extra approximation of the main text.

Working-set certificate. Figure S6 reports the working-set/capacity ratio over the near-optimal extra-traffic subset. This restriction matters: if one took the minimum peak unconstrained across all orders, a low-peak order with very high P2 cost could wrongly weaken the capacity-bound diagnosis. For Matmul_Case1/L1, the near-optimal ratio is 8.5, showing that the instance lies in a strongly capacity-bound region where the remaining optimization room comes mainly from the composition of spill cost, rather than from simply eliminating spills.

Objective misalignment between P1 and P2/P3. P1 targets lowering the per-cache peak, but §3.1 of the main text notes that a low peak is not equivalent to low spill cost. Figure S7 quantifies this misalignment:

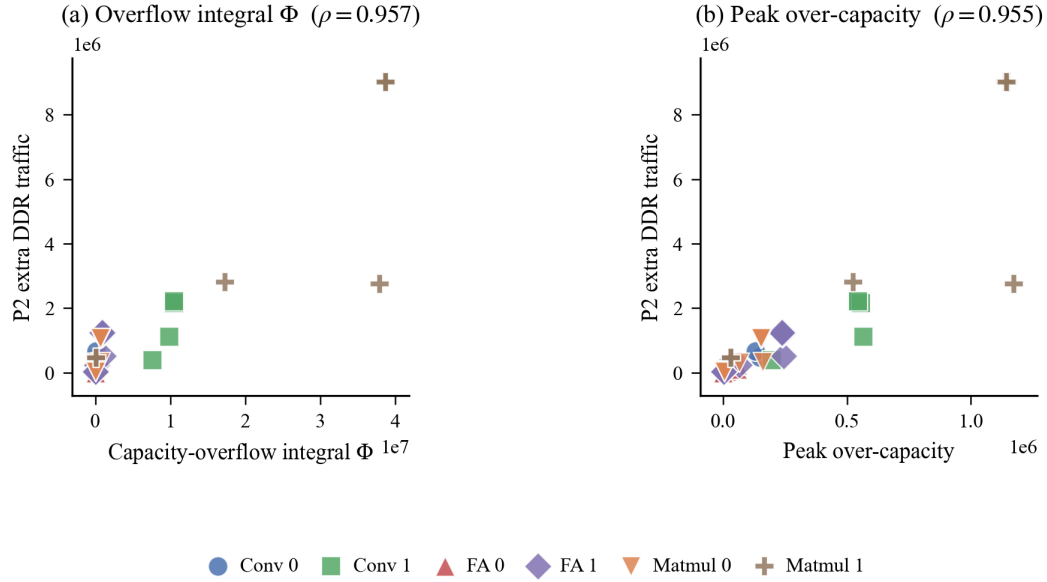


Figure S5: The overflow-area surrogate Φ against peak pressure. (a) Explanatory power of Φ for P2 extra traffic; (b) explanatory power of per-cache peak over-capacity for P2 extra traffic. Their correlations are close; the main role of Φ is to serve as a cheap and cross-stage-consistent selection signal.

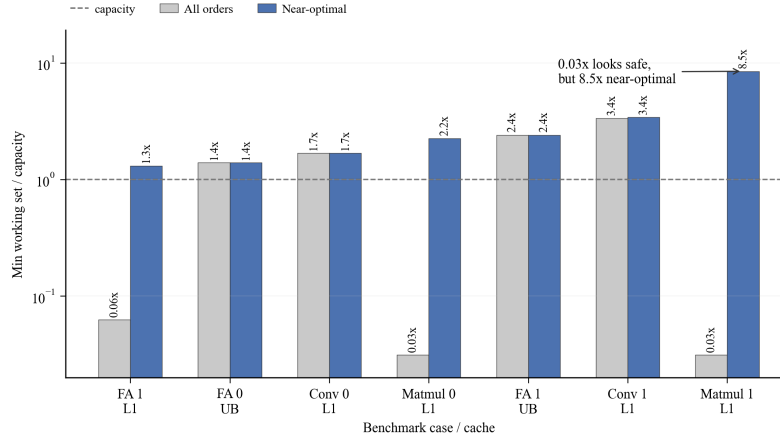


Figure S6: Working-set/capacity ratio over the near-optimal extra-traffic subset. The red entry shows that Matmul_Case1/L1 far exceeds capacity within the near-optimal set.

lowering the P1 objective alone does not guarantee feasibility under capacity pressure or low spill cost. On some instances, the P2 extra traffic of the P1-optimal order is in fact higher than that of the order our method selects, because P1 does not distinguish clean from dirty reserve: an order whose sole goal is a low peak may accumulate large amounts of dirty buffers in high-pressure windows, raising the actual eviction cost. The selection signal of the overflow-area surrogate Φ reflects multi-cache overflow pressure more directly, and so avoids this misalignment.

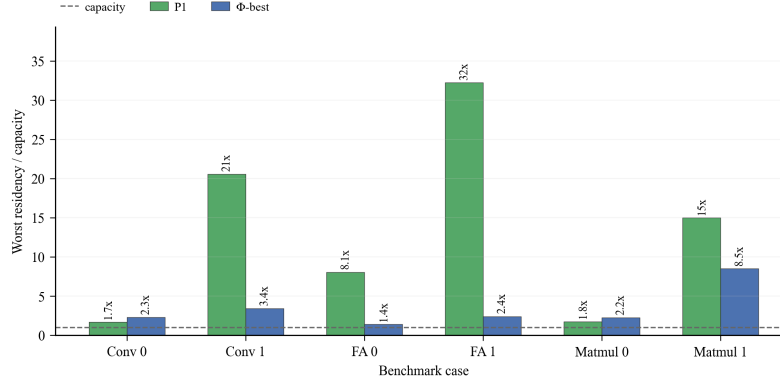


Figure S7: Misalignment between P1 and P2/P3. Lowering the P1 objective alone does not guarantee feasibility under capacity pressure or low spill cost; the Φ selection reflects multi-cache overflow pressure more directly.

Prefetch-window decoupling. Figure S8 shows the effect of the prefetch window H on P3 time and extra traffic. A nonzero window can noticeably reduce time, but extra traffic typically stays unchanged or rises slightly. This shows that the choice of spill set and the placement of reloads are two related but separable degrees of freedom: P2 must control traffic conservatively, whereas P3 can trade a wider window for pipeline overlap.

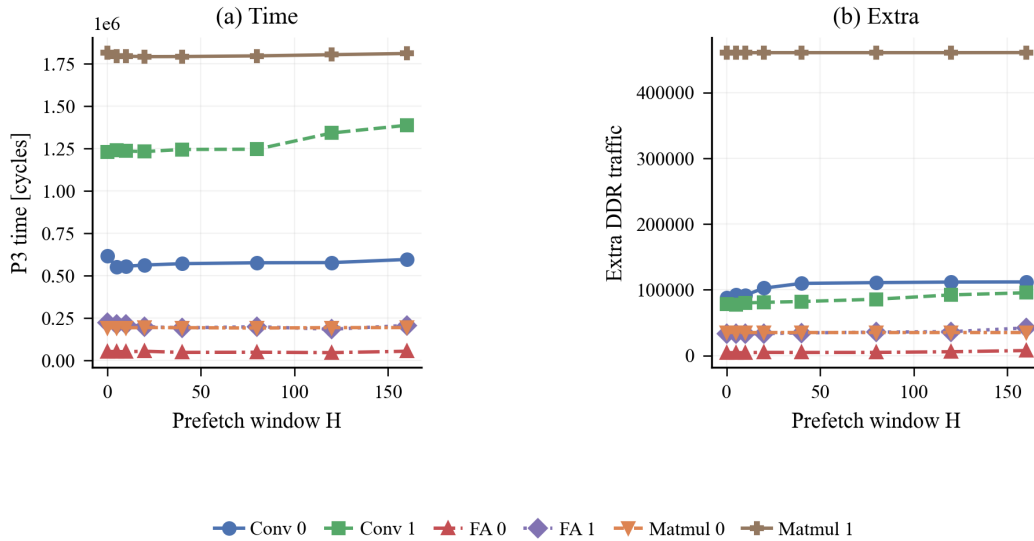


Figure S8: Effect of the prefetch window H on time and extra traffic. (a) P3 time as a function of H ; (b) extra traffic as a function of H . Prefetching mainly moves the placement of SPILL-IN, rather than redefining the spill set.

4.5 Controlled Clean/Dirty Ablation

The ablation in §4.3 of the main text reports the $2\times$ cost difference between clean reserve, with extra traffic 1,536, and dirty reserve, with extra traffic 3,072. This section gives the complete experimental design and results, ruling out a motif artifact of the public samples.

Figure S9 uses synthetic GEMM kernels to isolate the clean/dirty cost gradient. In (a), the DAG structure, peak, and spill count are fixed, and only the type of reserve available for eviction inside the high-pressure window is varied: the extra traffic of clean reserve is 1536 and that of dirty reserve is 3072, exactly $2\times$. In (b), sweeping a range of orders on the same clean instance shows that the higher the clean fraction in the peak residency, the lower the extra traffic. This experiment rules out the influence of the public-sample motif and directly validates the mechanism argument of the main text concerning the $2\times$ clean/dirty cost asymmetry.

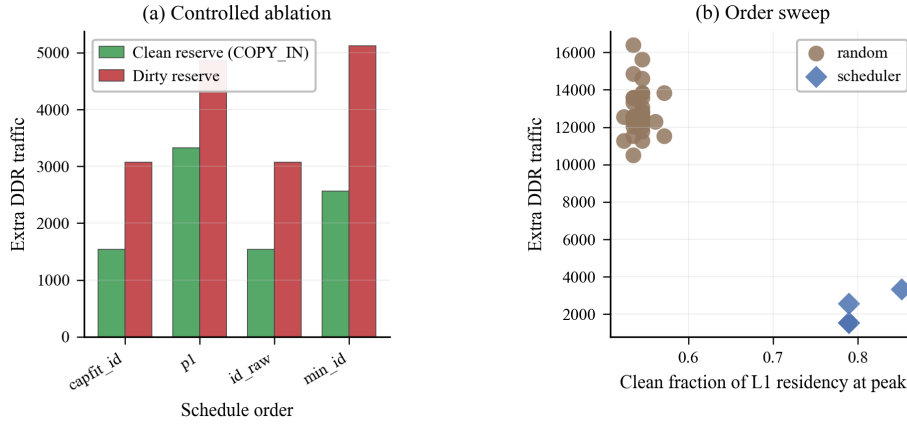


Figure S9: Clean/dirty mechanism validation on synthetic kernels. (a) The controlled ablation isolates the $2\times$ cost gradient; (b) extra traffic decreases as the clean fraction at the peak increases.